

实验九 卷积神经网络

（一）目的和要求

- （1）了解卷积神经网络与全连接神经网络的区别。
- （2）掌握卷积神经网络的结构。
- （3）理解卷积神经网络核心网络层的工作原理，包括卷积层、池化层、全连接层和 Dropout 层。
- （4）了解经典卷积神经网络的结构。

（二）实验知识准备

（1）什么是卷积神经网络

卷积神经网络（Convolutional Neural Network, CNN）是一种深度学习模型，特别适用于处理图像和视觉数据。以下是卷积神经网络的主要特点和组成部分：

1. **卷积层**，卷积操作：CNN 的核心是卷积层，通过卷积操作提取输入数据（如图像）中的特征。使用多个卷积核（滤波器）对输入进行滑动，生成特征图（feature map）。局部感受野，卷积操作只关注输入的局部区域，能够有效捕捉局部特征。
2. **池化层**，下采样：池化层通常跟随在卷积层后，用于减少特征图的尺寸，降低计算复杂度和内存占用，同时保持重要的特征信息。常见池化方法：最大池化（max pooling）和平均池化（average pooling）是最常用的池化技术。
3. **全连接层**，分类器：在卷积层和池化层之后，通常会有一个或多个全连接层，负责将提取到的特征映射到最终的输出类别，进行分类或回归任务。
4. **激活函数**，非线性变换：在卷积层和全连接层中，通常会使用非线性激活函数（如 ReLU）来引入非线性特性，使网络能够学习复杂的模式。
5. **特征自动学习**，CNN 不需要手动提取特征，能够自动从数据中学习到的有效的特征表示，特别适合于图像识别、物体检测等任务。
6. **应用领域**，计算机视觉：CNN 广泛应用于图像分类、目标检测、图像分割等任务。其他领域：除了图像数据，CNN 也被应用于音频处理、自然语言处理等领域。

（2）CNN 与 FCN 的区别

全连接神经网络在训练模型时，通常会因为全连接层的权重参数过多而导致

模型收敛和训练困难。受生物视觉系统的启发，卷积神经网络凭借着局部连接和权重共享特性在图像处理中脱颖而出，与全连接神经网络相比，卷积神经网络不仅减少了大量的训练参数，降低了神经网络的复杂度，减少了模型的过拟合程度。

1. 局部连接

传统的前馈神经网络，前一层的结点与后一层的结点全部连接起来，这种连接方式称为全连接，如图所示。如果前一层有 m 个结点，后一层有 n 个结点，就会有 $m \times n$ 个连接权重，完成一次反向传播更新权重时，要对这些权重进行重新计算，造成了 $O(m \times n) = O(n^2)$ 的计算开销和存储开销。图 1 所示。

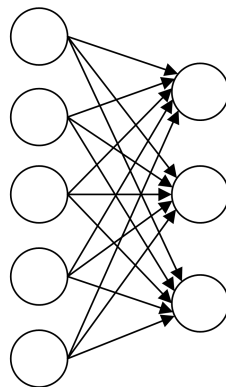


图 1 全连接的神经网络

而局部连接的思想是两层之间只有相邻的结点才进行连接，即连接都是“局部”的，如图 2 所示。以图像处理为例，图像的某一个局部的像素组合在一起共同呈现一些特征，而图像中距离比较远的像素组合起来则没有实际意义，因此这种局部连接的方式可以在图像处理问题上有较好的表现。如果把连接限制在空间中相邻的 C 个结点，就可把连接权重降低至 $C \times n$ ，计算开销和存储开销就降低至 $O(C \times n) = O(n)$ 。

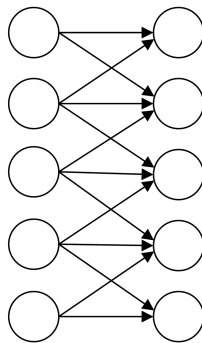


图 2 局部连接的神经网络

2. 权重共享

在全连接神经网络中，当计算从输入层经隐藏层到输出层，权重矩阵中的每个元素都只使用过一次，即每个元素与输入矩阵中的一个元素相乘，之后不会再重复利用。针对这一点，卷积神经网络是在输入图像的不同位置使用同一个卷积核进行计算。对于每一个卷积核，它的核内参数在计算过程中都是不变的。

(3) CNN 结构

典型的卷积神经网络由卷积层、池化层和全连接层 3 部分组成。在图像分类中表现良好的深度卷积神经网络，往往由多个“卷积层+池化层”的组合堆叠而成，通常多达十几层甚至上百层，如图 3 所示。

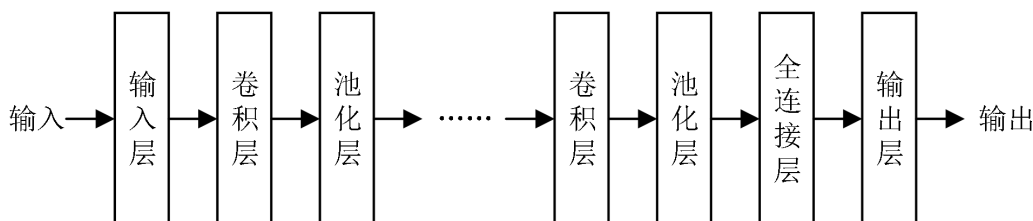


图 3 卷积神经网络结构

其中，卷积层用于提取图像中的局部特征；池化层用于对提取出的局部特征进行降维处理，防止过拟合；全连接层用于接收池化层的输出，为后续分类任务做准备。

(4) 经典的 CNN

1. LeNet-5

LeNet-5 是杨立昆在 1998 年设计的卷积神经网络。初期，LeNet-5 主要用于手写数字识别任务，其在 MNIST 数据集上的识别率可以达到 99.2%。LeNet-5 是最基本的卷积神经网络算法，被认为是卷积神经网络的开篇之作。

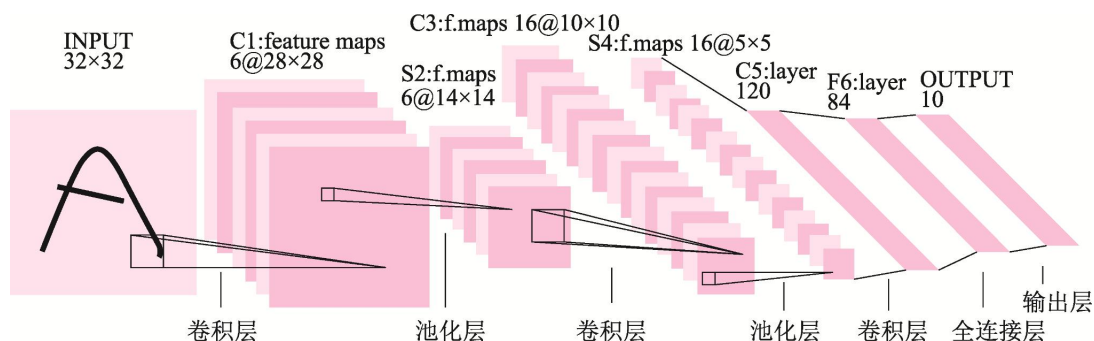


图 4 LeNet-5 网络结构

LeNet-5 的网络结构是：输入层→卷积层→池化层→卷积层→池化层→卷积

层→全连接层→输出层。各层说明图表 1 所示。

表 1 各层说明

层结构	说明
输入层	输入为 32×32 的灰度图像。
卷积层 C1	该层将输入矩阵与 6 个大小为 5×5 的卷积核进行 VALID 卷积运算，输出的特征图的尺寸为 28×28，深度为 6。
池化层 S2	该层采用 VALID 最大池化模式，池化窗口的尺寸为 2×2，经池化后得到 6 个 14×14 的特征图。
卷积层 C3	该层将输入与 16 个大小为 5×5 的卷积核进行 VALID 卷积运算，输出为 16 个 10×10 的特征图。
池化层 S4	该层仍采用 VALID 最大池化模式，池化窗口的尺寸为 2×2，经池化后得到 16 个 5×5 的特征图。
卷积层 C5	该层将输入与 120 个大小为 5×5 的卷积核进行 VALID 卷积运算，输出为 120 个 1×1 的特征图。
全连接层 F6	该层与 C5 层全连接，有 84 个神经元。
输出层	由于 LeNet-5 最初用于手写数字识别，处理的是 0~9 的 10 分类问题，因此，该层有 10 个输出。

2. VGGNet

VGGNet 是由牛津大学计算机视觉组和谷歌 DeepMind 团队的研究人员共同研发提出的深度卷积神经网络，夺得了 2014 年 ILSVRC 竞赛分类项目的第二名和定位项目的第一名。VGGNet 成功地构建出了深达 19 层的卷积神经网络模型，探索了卷积神经网络的深度和模型性能之间的关系，即证明了卷积神经网络的深度对网络模型性能的提高起着关键性的作用。如图 5 所示

ConvNet 配置					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
输入层 (224×224 RGB 图像)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
最大池化层					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
最大池化层					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256

图 5 VGGNET 结构

图 5 表明 VGGNet 模型有 VGG11、VGG13、VGG16 和 VGG19 等 6 种网络结构，每种结构都含有 5 组卷积，每组卷积都使用 3×3 的卷积核，每组卷积后进行 2×2 的最大池化，接下来是 3 个全连接层。

最大池化层					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
最大池化层					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
最大池化层					
全连接层-4096					
全连接层-4096					
全连接层-1000					
激活函数 (Softmax 函数)					

图 5 VGGNET 结构(续表)

在 VGGNet 结构中，最广为流传的两种结构是 VGG16（见图 5 的网络结构 D）和 VGG19（见图 5 的网络结构 E）。

VGG16 包含了 16 个隐藏层（13 个卷积层和 3 个全连接层），VGG19 包含了 19 个隐藏层（16 个卷积层和 3 个全连接层），两者并没有本质上的区别，只是网络深度不同。在训练高级别的网络时，可以先训练低级别的网络，用前者获得的权重初始化高级别的网络，可以加速网络的收敛。

3. ResNet

ResNet 由微软亚洲研究院何恺明等人于 2015 年提出，目前已经成为运用最为广泛的卷积神经网络之一。与 LeNet-5、VGGNet 等卷积神经网络相比，ResNet 凭借着独有的残差学习网络结构在其中脱颖而出。网络结构如图 6 所示。

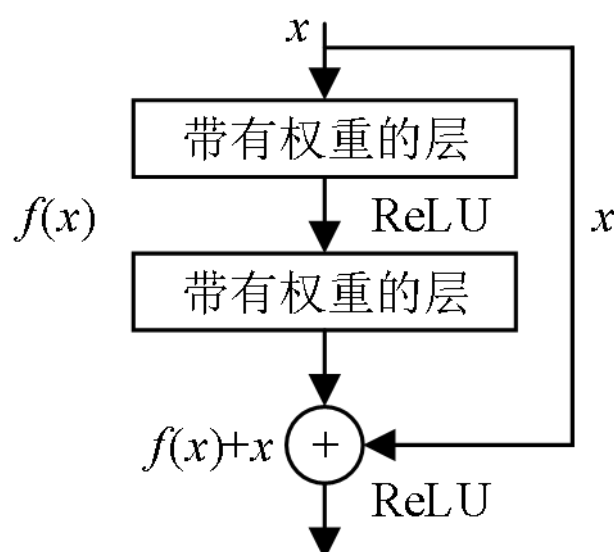


图 6 ResNET 网络结构

ResNet 的残差学习模块包含多个卷积层， x 表示这个模块接收到的数据。 x 经过多个卷积层处理，同时 x 跳过多个卷积层直接传导到后面的层中，将 x 的值与最后一个卷积层的输出特征图相加，相加后的特征图经过 ReLU 函数处理，作为下一层的输入。这样就保证了下一层既能够接收前面多个卷积层处理后的数据，还能接收 x 。

（三）实验内容及步骤

（1）基于 Cifar-10 数据集的彩色图像识别分类

Cifar-10 数据集是图像识别分类数据集，共分为 10 类，类别包括飞机、狗、船等。该数据集包含了 60 000 张 32×32 的彩色图像，其中训练数据集 50000 张，测试数据集 10000 张。

1. 步骤一，导入本项目所需要的模块和包，从 Keras 中导入 Cifar-10 数据集，将训练集的特征值和标签值分别存储在 `x_train` 和 `y_train` 中，将测试集的特征值和标签值分别存储在 `x_test` 和 `y_test` 中。将特征值 `x_train` 和 `x_test` 的数据类型转换为 `tf.float32`，并进行标准化处理，使其取值范围为 0~1；将标签值 `y_train` 和 `y_test` 的数据类型转换为 `tf.int32`。如图 1 所示。

```
1 #导入所需要的模块与包
2 import tensorflow as tf
3 import numpy as np
4 import matplotlib.pyplot as plt
5 #导入Cifar-10数据集，将训练集和测试集存储在相应变量中
6 cifar10=tf.keras.datasets.cifar10
7 (x_train,y_train),(x_test,y_test)=cifar10.load_data()
8 #转换特征值的数据类型并进行标准化处理
9 x_train,x_test=tf.cast(x_train,dtype=tf.float32)/255.0,tf.cast(x_test,dtype=tf.float32)/255.0
10 #转换标签值的数据类型
11 y_train,y_test=tf.cast(y_train,dtype=tf.int32),tf.cast(y_test,dtype=tf.int32)
12 #显示数据集的特征值和标签值的shape属性值
13 print("x_train.shape=",x_train.shape)
14 print("y_train.shape=",y_train.shape)
15 print("x_test.shape=",x_test.shape)
16 print("y_test.shape=",y_test.shape)
```

图 1 准备数据集

2. 步骤二，构建 CNN 模型，图像识别分类网络模型采用卷积神经网络，包括两个卷积层、两个池化层、4 个 Dropout 层、一个拉伸层、两个全连接层和一个输出层，网络结构如图 2 所示。

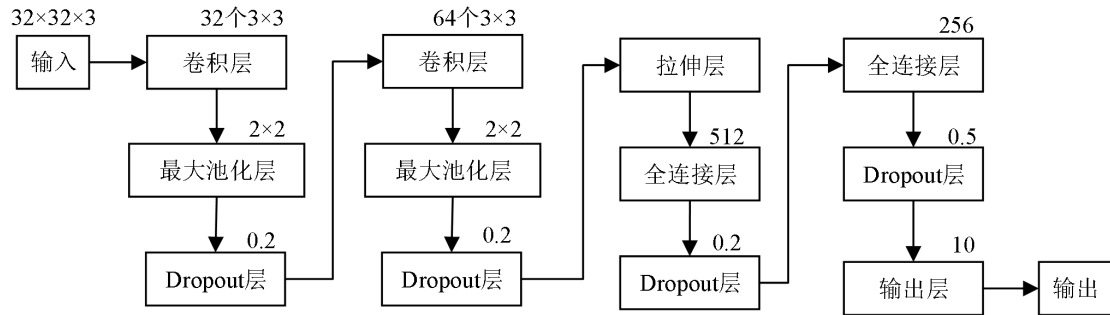


图 2 图像识别分类网络结构

主要代码如图 3 所示。

```

1 #构建网络模型
2 model=tf.keras.models.Sequential([
3 #创建卷积层
4 tf.keras.layers.Conv2D(32, kernel_size=(3, 3), padding='SAME', activation=tf.nn.relu, input_shape=x_train.shape[1:]),
5 tf.keras.layers.MaxPool2D(pool_size=(2, 2), strides=(1, 1), padding='SAME'), #创建最大池化层
6 tf.keras.layers.Dropout(0.2), #创建Dropout层
7 tf.keras.layers.Conv2D(64, kernel_size=(3, 3), padding='SAME', activation=tf.nn.relu), #创建卷积层
8 tf.keras.layers.MaxPool2D(pool_size=(2, 2), strides=(1, 1), padding='SAME'), #创建最大池化层
9 tf.keras.layers.Dropout(0.2), #创建Dropout层
10 tf.keras.layers.Flatten(), #创建拉伸层
11 #创建全连接层
12 tf.keras.layers.Dense(512, activation='relu'),
13 tf.keras.layers.Dropout(0.2), #创建Dropout层
14 #创建全连接层
15 tf.keras.layers.Dense(256, activation='relu'),
16 tf.keras.layers.Dropout(0.5), #创建Dropout层
17 #创建全连接层作为输出层
18 tf.keras.layers.Dense(10, activation='softmax')
19 ])
20 model.summary()
  
```

图 3 构建 CNN 网络模型

图 3 所示代码中，构建卷积神经网络模型对象 model。在 model 对象中添加第一组“卷积层+池化层+Dropout 层”，包括添加卷积层，输出通道数为 32，卷积核大小为 3×3，填充方式为'SAME'，使用 ReLU 函数作为激活函数；然后添加最大池化层，池化窗口大小为 2×2，步长大小为(1,1)，填充方式为'SAME'；最后添加 Dropout 层，丢弃概率为 0.2。在 model 对象中添加第二组“卷积层+池化层+Dropout 层”，包括添加卷积层，输出通道数为 64，卷积核大小为 3×3，填充方式为'SAME'，使用 ReLU 函数作为激活函数；然后添加最大池化层，池化窗口大小为 2×2，步长大小为(1,1)，填充方式为'SAME'；最后添加 Dropout 层，丢弃概率为 0.2。

在 model 对象中添加拉伸层。在 model 对象中添加全连接层，神经元个数为 512，使用 ReLU 函数作为激活函数。在 model 对象中添加 Dropout 层，丢弃概

率为 0.2。在 model 对象中添加全连接层，神经元个数为 256，使用 ReLU 函数作为激活函数。在 model 对象中添加 Dropout 层，丢弃概率为 0.5。在 model 对象中添加输出层，Cifar-10 数据集是 10 分类问题，因此输出层有 10 个神经元，激活函数采用 Softmax 函数。

3. 步骤三，编译、训练和评估 CNN 模型，主要代码如图 4 所示。

```
1 #编译网络模型
2 model.compile(optimizer='adam', loss=tf.keras.losses.SparseCategoricalCrossentropy(), metrics=
  ['sparse_categorical_accuracy'])
3 #训练网络模型
4 history = model.fit(x_train, y_train, batch_size=128, epochs=10, validation_split=0.2)
5 #评估网络模型
6 model.evaluate(x_test, y_test, batch_size=64, verbose=2)
```

图 4 编译、训练和评估 CNN 模型

图 3 所示代码作用为编译网络模型，其中，优化器使用 Adam 优化器，损失函数使用稀疏交叉熵损失函数，性能评估函数使用稀疏交叉熵准确率函数。训练网络模型，并将训练结果保存在 history 中。其中，设置批量的大小为 128，迭代次数为 10，验证集的数据占比为 0.2。使用测试集数据对网络模型进行评估，其中，设置批量的大小为 64，日志显示模式为 2。训练过程如图 5：

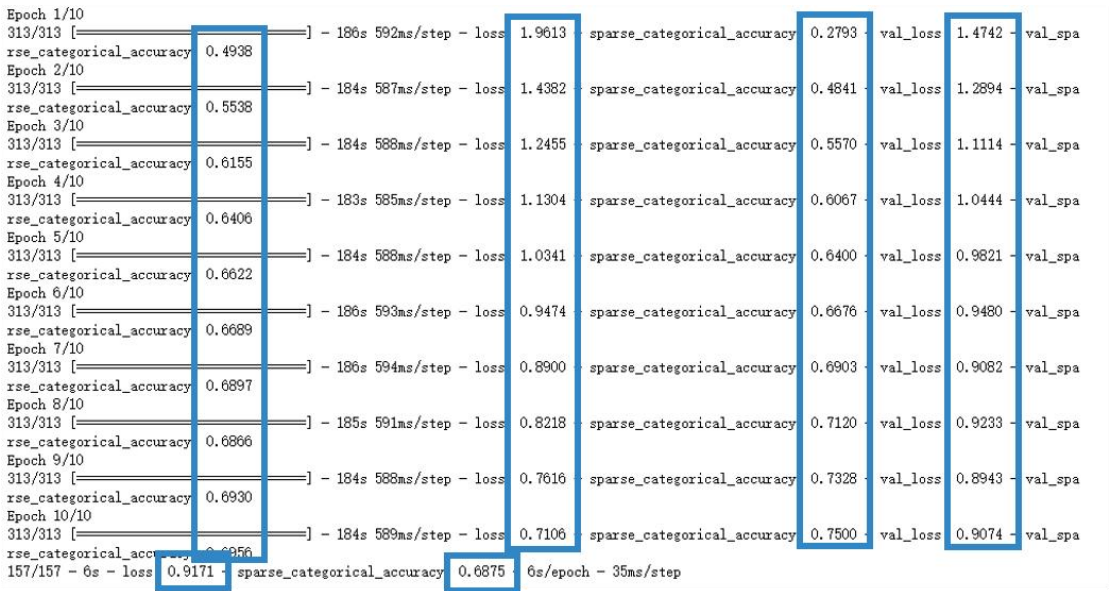


图 5 训练过程可视化

从结果中可以看出，训练集损失函数值比验证集和测试集更低，训练集的准确率也比验证集和测试集更高；测试集损失函数值约为 0.9171，准确率约为 68.75%。

4. 步骤四，可视化训练的结果，如图 6 所示。

```
1 #读取history的history属性
2 loss=history.history['loss']           #训练集损失函数值
3 #训练集准确率
4 acc=history.history['sparse_categorical_accuracy']
5 val_loss=history.history['val_loss']    #验证集损失函数值
6 #验证集准确率
7 val_acc=history.history['val_sparse_categorical_accuracy']
8 #创建画布并设置画布的大小
9 plt.figure(figsize=(10,3))
10 #在子图1中绘制损失函数值的折线图
11 plt.subplot(121)
12 plt.plot(loss,color='b',label='train')
13 plt.plot(val_loss,color='r',label='validate')
14 plt.ylabel('loss')
15 plt.legend()
16 #在子图2中绘制准确率的折线图
17 plt.subplot(122)
18 plt.plot(acc,color='b',label='train')
19 plt.plot(val_acc,color='r',label='validate')
20 plt.ylabel('Accuracy')
21 plt.legend()
22 plt.show()
```

图 6 结果可视化

上述代码中，首先读取 history 的 history 属性，分别将训练集损失函数值赋值给变量 loss、训练集准确率赋值给变量 acc、验证集损失函数值赋值给变量 val_loss、验证集准确率赋值给变量 val_acc。最后创建子图，在子图 1 中绘制损失函数值的折线图。在子图 2 中绘制准确率的折线图，如图 7 所示。

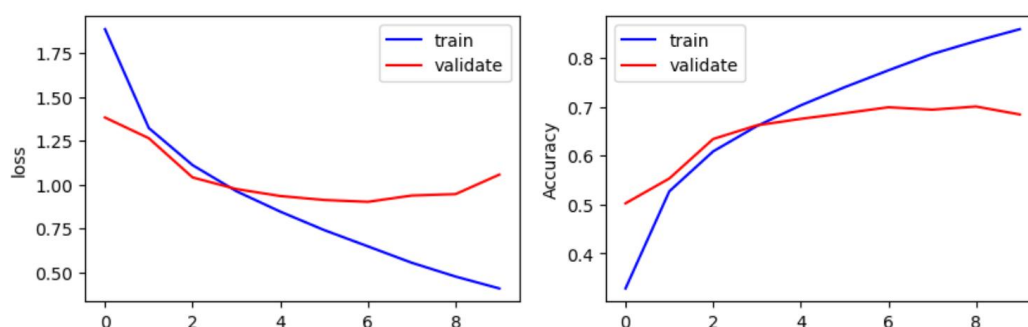


图 7 CNN 损失函数值和准确率的折线图

从结果中可以看出，随着训练迭代次数的增加，训练集的损失函数值降低速率高于验证集，同比训练集的准确率也高于验证集，说明虽然在训练网络模型中加入了 Dropout 层来避免过拟合现象的发生，但网络模型与训练集的拟合度更好。

5. 步骤五，应用卷积神经网络模型，主要代码如图 8 所示。

```
1 plt.figure()
2 for i in range(10):
3     n=np.random.randint(1,10000)           #生成随机整数n
4     plt.subplot(2,5,i+1)                   #创建子图
5     plt.axis("off")                        #设置不显示坐标轴
6     plt.rcParams['font.sans-serif']=['SimHei']
7     plt.imshow(x_test[n],cmap='gray')      #绘制图像
8     demo=tf.reshape(x_test[n],(1,32,32,3))
9     y_pred=np.argmax(model.predict(demo))   #应用网络模型
10    title="标签值: "+str((y_test.numpy())[n,0])+"\n预测值: "+str(y_pred)
11    plt.title(title)                        #设置子图标题
12 plt.show()
```

图 8 应用模型主要代码

上图代码为在测试集中随机选择 10 张图片，使用训练完成的卷积神经网络模型进行预测。主要步骤为：使用随机函数生成随机整数赋值给变量 n，n 代表测试集中选中数据的下标。将该数据平展为一维数组并存入 x。应用网络模型得到概率最大的下标，即预测值。创建子图，在子图中绘制测试图像，并在子图标题上显示标签值和预测值。显示图形。结果如图 9 所示。



图 9 应用模型结果

从结果中可以看出，有 3 张图像的预测值与标签值不一致，说明网络模型预测错误。为了提高网络模型的准确率，可以采用加深网络模型深度、增加训练轮数、调整学习率等方法。

(2) 实训项目

参考教材 P240-253，完成基于 CNN 的手写数字识别的实验。